Experiment Number: 01

Experiment Name: Namespace and overloading.

Objectives:

- To implement a simple class Number with operator overloading for + and *
- To organize the class inside a namespace and use alias for better code management.

Theory:

Namespace is a feature in C++ that groups related classes, functions, and variables under a named scope. It prevents name conflicts in large programs. Example: namespace miens { class Number { ... }; }. We can access members using mynas::Number or create an alias like namespace man = mines;.

Operator overloading allows user-defined types (like classes) to use operators (+, *, etc.) like built-in types. It is done by defining member or friend functions. Example: Number operator+(const Number& other) const for addition. For multiplication with int on either side, we use member function for right-side and friend for left-side.

Algorithm:

1. Define class Number with private int data and constructor.
2. Implement value( ) getter.
3. Overload + .
4. Overload * .
5. Place class in myns namespace.
6. In main( ), use alias mn = myns,
7. Create objects, perform operations, and print results using value().

Code:

here | *Namespace_Assaignment.cpp

```cpp
#include <iostream>
using namespace std;
namespace myns {
class Number {
  private:
    int data;
  public:
    Number(int d = 0) {
      data = d; }
    int value( ) const {
      return data; }
    Number operator+(const Number& other) const {
      return Number(data + other.data); }
    Number operator*(const Number& other) const {
      return Number(data * other.data); }
    Number operator*(int num) const {
      return Number(data * num); }
    friend Number operator*(int num, const Number& n);
};
  Number operator*(int num, const Number& n) {
    return Number(num * n.data); }}
int main( ) {
  namespace mn = myns;
  mn::Number a(5);
  cout << "a = " << a.value( ) << endl;
  mn::Number b(3);
  mn::Number c = a + b;
  cout << "a + b = " << c.value( ) << endl;
  mn::Number c1 = a * b;
  mn::Number c2 = a * 2;
  mn::Number c3 = 2 * a;
  cout << "a * b = " << c1.value( ) << endl;
  cout << "a * 2 = " << c2.value( ) << endl;
  cout << "2 * a = " << c3.value( ) << endl;
  mn::Number x(2), y(4);
  mn::Number sum = x + y;
  cout << "x + y = " << sum.value( ) << endl;
  return 0;
}
```

Output:

```
a = 5
a + b = 8
a * b = 15
a * 2 = 10
2 * a = 10
x + y = 6
```

Conclusion:

In this experiment we have used namespace and overloading, the code successfully overloads + and * operators, allowing natural syntax like a + b or 2 * a. Namespace myns with alias mn keeps code clean and avoids conflicts.

Experiment Number: 02

Experiment Name: Operator Overloading .

Objectives:

- To understand how Operator Overloading work in C++.
- To overload binary, unary, and relational operators for a Number class.

Theory:

Redefine or customize the standard behavior of operators is operator overloading. There are 3 types of overloading :

- Binary Operator: which takes two operands and returns a new number.
- Unary Operator: Which takes one operand and increments or decrements it.
- Relational Operator: Which compares 2 numbers and returns true or false.

Algorithm:

1. Define Number with int x, y, constructor, and show( ).
2. Overload < obj.x && y < obj.y).
3. Overload -: create temp, temp.x = x - obj.x, return temp.
4. Overload ++: save current in temp, increment x and y, return temp.
5. In main( ): copy, compare, subtract, increment, display

Code:

ere × Namespace_Assaignment.cpp × Operator_Overloading.cpp ×

```
1   #include<iostream>
2   using namespace std;
3   class Number {
4   public:
5       int x, y;
6       Number(int p, int q){
7           x = p;
8           y = q;
9       }
10
11      int operator < (Number obj) {
12          if (x < obj.x && y < obj.y) {
13              return 1;
14          }
15          else return 0;
16      }
17      Number operator - (Number obj) {
18          Number temp(0,0);
19          temp.x = x - obj.x;
20          temp.y = y - obj.y;
21          return temp;
22      }
23      Number operator++( ) {
24          Number temp(0,0);
25          temp.x = x;
26          temp.y = y;
27          x = x + 1;
```

ere × Namespace_Assaignment.cpp × Operator_Overloading.cpp ×

```
24          Number temp(0,0);
25          temp.x = x;
26          temp.y = y;
27          x = x + 1;
28          y = y + 1;
29          return temp;
30      }
31
32      void show( ) {
33          cout << x << " , " << y <<endl;
34      }
35  };
36  int main( ) {
37      Number n1(0,0), n2(1,5), n3(5,10);
38      n1 = n2;
39      if (n1 < n3) {
40          n2 = n3 - n1;
41          ++n1;
42      }
43      else cout<<"n1 is greater then n3";
44      cout << "Substracton: ";
45      n2.show( );
46      cout<<"Incrementing n1: ";
47      n1.show( );
48  }
49
```

Output:

Conclusion:

In this experiment the Number class overloads binary, unary, and relational operators. All operators work correctly: - subtracts components, < checks dominance, ++ follows pre-increment with old-value return. Code is clean and demonstrates operator overloading effectively.